Algorytmy macierzowe

Sprawozdanie – Ćwiczenie 1

Mnożenie macierzy

Autorzy:
Wiktor Augustyn
Gabriel Wermiński

Grupa: 2
Data: 15.10.2026

Specyfikacja sprzętowa

Pomiary wykonano na laptopie Dell Inspiron 14 5420.

- Procesor: 12th Gen Intel(R) Core(TM) i5-1235U (4+8) @ 4.40 GHz.
- Pamięć operacyjna: 16 GB.
- System: EndeavourOS, Linux 7.2.9-arch1-1

Opis algorytmów

W obu algorytmach zakładamy, że macierze są kwadratowe o rozmiarze $n \times n$, gdzie n jest potęgą dwójki (w przeciwnym razie macierze uzupełniamy zerami). Dla $n \leq n_0$ stosujemy klasyczne mnożenie.

Rekurencyjne mnożenie metodą Bineta

1 **funkcja** Binet(A, B, n)
2 **jeśli** $n \leq n_0$ **to zwróć** $A \cdot B$ (mnożenie klasyczne)
3 $k \leftarrow n/2$
4 podziel A na bloki $A_{11}, A_{12}, A_{21}, A_{22}$ o rozmiarze $k \times k$
5 podziel B na bloki $B_{11}, B_{12}, B_{21}, B_{22}$ o rozmiarze $k \times k$
6 $C_{11} \leftarrow \text{Binet}(A_{11}, B_{11}, k) + \text{Binet}(A_{12}, B_{21}, k)$
7 $C_{12} \leftarrow \text{Binet}(A_{11}, B_{12}, k) + \text{Binet}(A_{12}, B_{22}, k)$
8 $C_{21} \leftarrow \text{Binet}(A_{21}, B_{11}, k) + \text{Binet}(A_{22}, B_{21}, k)$
9 $C_{22} \leftarrow \text{Binet}(A_{21}, B_{12}, k) + \text{Binet}(A_{22}, B_{22}, k)$
10 złóż C z bloków $C_{11}, C_{12}, C_{21}, C_{22}$
11 **zwróć** C

Algorytm 1: Rekurencyjne mnożenie macierzy metodą Bineta

Algorytm wykonuje 8 rekurencyjnych mnożeń macierzy rozmiaru $n/2$, więc $T(n) = 8T(n/2) + \Theta(n^2)$, czyli złożoność wynosi $\Theta(n^3)$.

Rekurencyjne mnożenie metodą Strassena

funkcja Strassen(A, B, n)
  jeśli $n \leq n_0$ **to zwróć** $A \cdot B$ (mnożenie klasyczne)
  $k \leftarrow n/2$
  podziel A na bloki $A_{11}, A_{12}, A_{21}, A_{22}$ o rozmiarze $k \times k$
  podziel B na bloki $B_{11}, B_{12}, B_{21}, B_{22}$ o rozmiarze $k \times k$
  $M_1 \leftarrow \text{Strassen}(A_{11} + A_{22}, B_{11} + B_{22}, k)$
  $M_2 \leftarrow \text{Strassen}(A_{21} + A_{22}, B_{11}, k)$
  $M_3 \leftarrow \text{Strassen}(A_{11}, B_{12} - B_{22}, k)$
  $M_4 \leftarrow \text{Strassen}(A_{22}, B_{21} - B_{11}, k)$
  $M_5 \leftarrow \text{Strassen}(A_{11} + A_{12}, B_{22}, k)$
  $M_6 \leftarrow \text{Strassen}(A_{21} - A_{11}, B_{11} + B_{12}, k)$
  $M_7 \leftarrow \text{Strassen}(A_{12} - A_{22}, B_{21} + B_{22}, k)$
  $C_{11} \leftarrow M_1 + M_4 - M_5 + M_7$
  $C_{12} \leftarrow M_3 + M_5$
  $C_{21} \leftarrow M_2 + M_4$
  $C_{22} \leftarrow M_1 - M_2 + M_3 + M_6$
  złóż C z bloków $C_{11}, C_{12}, C_{21}, C_{22}$
  zwróć C

Algorytm 2: Rekurencyjne mnożenie macierzy metodą Strassena

Algorytm wykonuje 7 rekurencyjnych mnożeń zamiast 8 (kosztem większej liczby dodawań), więc $T(n) = 7T(n/2) + \Theta(n^2)$, czyli złożoność wynosi $\Theta\left(n^{\log_2 7}\right) \approx \Theta(n^{2,81})$.

Na wykresach w następnym rozdziale linia przerywana i kropkowana to eksperymentalne dopasowanie prostej i paraboli do zmierzonych wyników Bineta i Strassena. To tylko trend. Oszacowaniem złożoności pozostają rekurencje powyżej, bo liczba operacji rośnie szybciej niż kwadrat.

Fragmenty implementacji

Oba algorytmy dopełniają wejście zerami do najbliższej potęgi dwójki, a rekurencja schodzi do jednego mnożenia skalarnego.

Dla Bineta każdy krok rozcina macierz na cztery bloki i wykonuje osiem mnożeń rekurencyjnych. Dodanie dwóch bloków jest liczone jako tyle operacji zmiennoprzecinkowych, ile ma elementów wynik.

```
n = next_pow2(max(m_A, n_A, m_B, n_B))
A_padded = np.pad(A, ((0, n - m_A), (0, n - n_A)), mode="constant")
B_padded = np.pad(B, ((0, n - m_B), (0, n - n_B)), mode="constant")

if A_i_e - A_i_s == 1:
    m_ops += 1
    return np.array([[A_padded[A_i_s][A_j_s] * B_padded[B_i_s][B_j_s]]])

C_11 = (rec(*se_11_A[0], *se_11_A[1], *se_11_B[0], *se_11_B[1]) +
        rec(*se_12_A[0], *se_12_A[1], *se_21_B[0], *se_21_B[1]))
a_s_ops += C_11.size
C_12 = (rec(*se_11_A[0], *se_11_A[1], *se_12_B[0], *se_12_B[1]) +
```

```
        rec(*se_12_A[0], *se_12_A[1], *se_22_B[0], *se_22_B[1]))
a_s_ops += C_12.size
C_21 = (rec(*se_21_A[0], *se_21_A[1], *se_11_B[0], *se_11_B[1]) +
        rec(*se_22_A[0], *se_22_A[1], *se_21_B[0], *se_21_B[1]))
a_s_ops += C_21.size
C_22 = (rec(*se_21_A[0], *se_21_A[1], *se_12_B[0], *se_12_B[1]) +
        rec(*se_22_A[0], *se_22_A[1], *se_22_B[0], *se_22_B[1]))
a_s_ops += C_22.size
return np.block([[C_11, C_12], [C_21, C_22]])
```

Strassen w każdym kroku liczy siedem iloczynów pomocniczych, a dodawania i odejmowania bloków też wchodzą do licznika operacji.

```
M1 = rec(add(A11, A22), add(B11, B22))
M2 = rec(add(A21, A22), B11)
M3 = rec(A11, sub(B12, B22))
M4 = rec(A22, sub(B21, B11))
M5 = rec(add(A11, A12), B22)
M6 = rec(sub(A21, A11), add(B11, B12))
M7 = rec(sub(A12, A22), add(B21, B22))

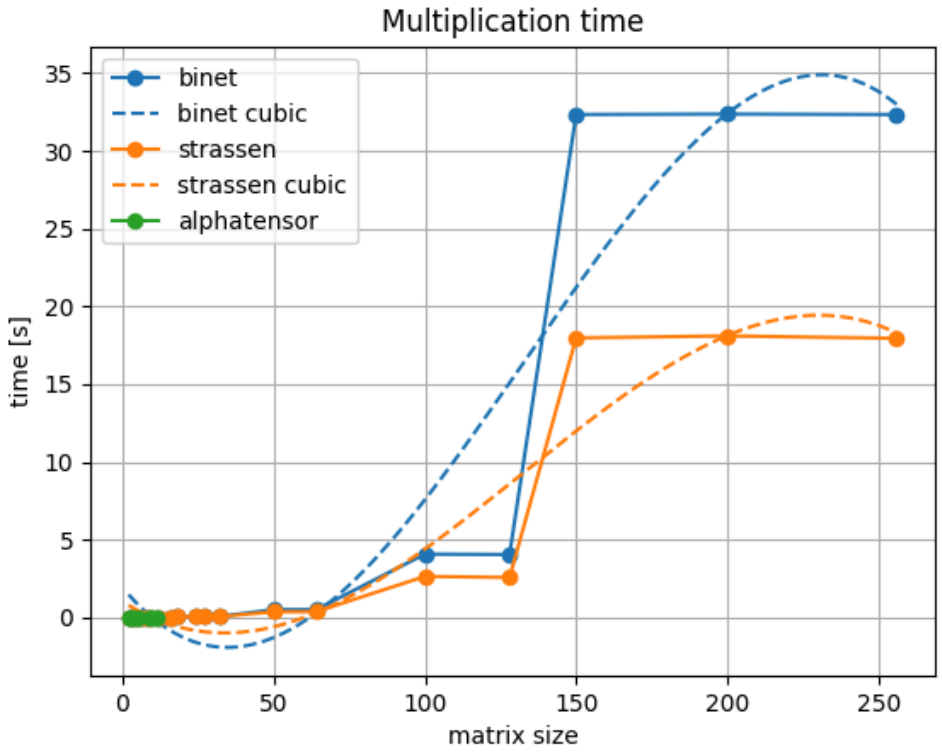
C11 = add(sub(add(M1, M4), M5), M7)
C12 = add(M3, M5)
C21 = add(M2, M4)
C22 = add(sub(add(M1, M3), M2), M6)
```

Wyniki pomiarów

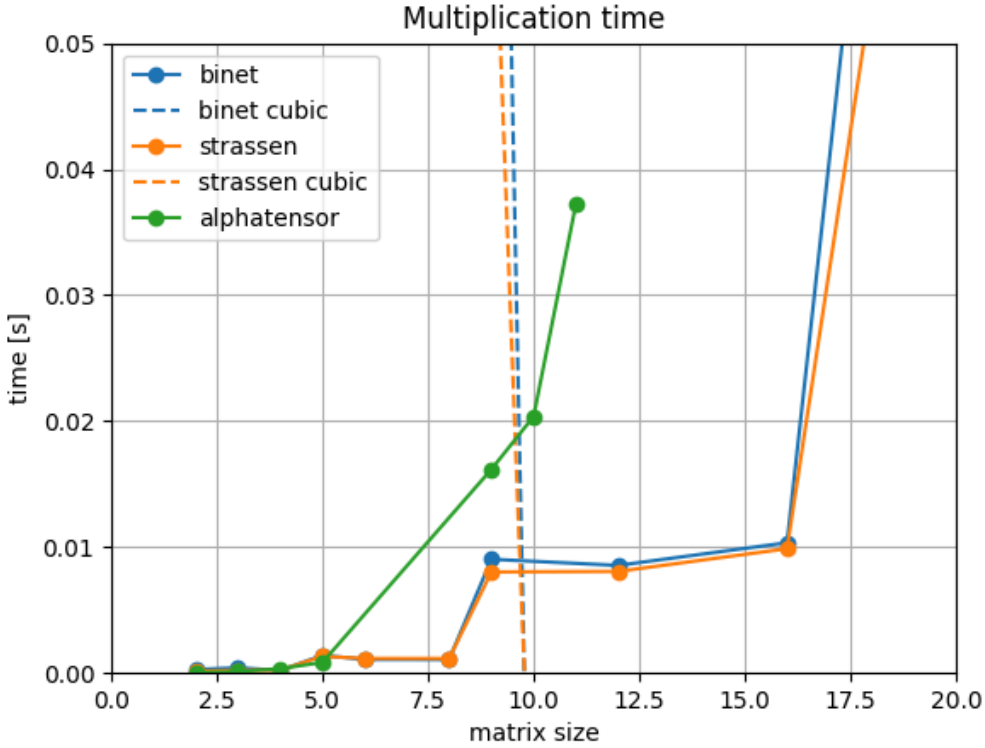
Zmierzono rozmiary od 2 do 100. Rozmiar 1 jest przypadkiem bazowym: jedno mnożenie skalarne. Rozmiar 100 jest największym, który ta implementacja w czystym Pythonie kończy w akceptowalnym czasie. Dla $n = 1000$ macierz jest dopełniana do 1024, a sam Binet wykonuje wtedy 1024^3 mnożeń skalarnych w interpreterze, więc ten rozmiar nie jest praktycznie obliczalny.

Operacje zmiennoprzecinkowe liczymy jako sumę mnożeń oraz dodawań i odejmowań. AlphaTensor występuje tylko dla rozmiarów, dla których plik rozkładów ma algorytm: 2, 3, 4, 5, 9, 10 i 11. Próbka jest za mała na dopasowanie krzywej.

Czas wykonania w zależności od rozmiaru macierzy



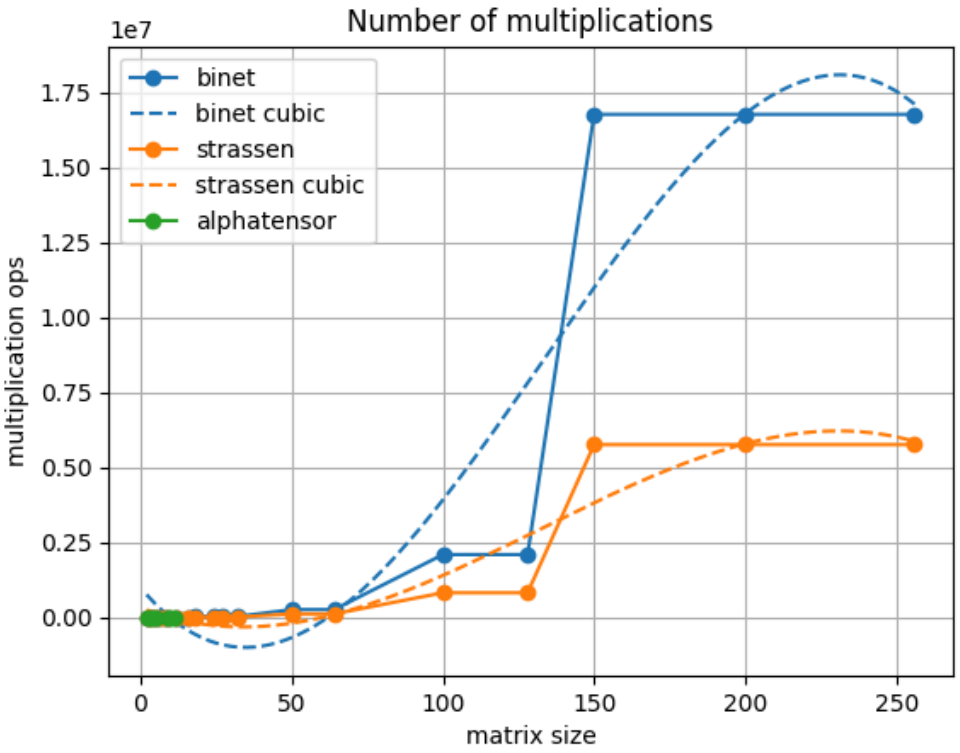
Rysunek 1: Czas wykonania w zależności od rozmiaru macierzy n



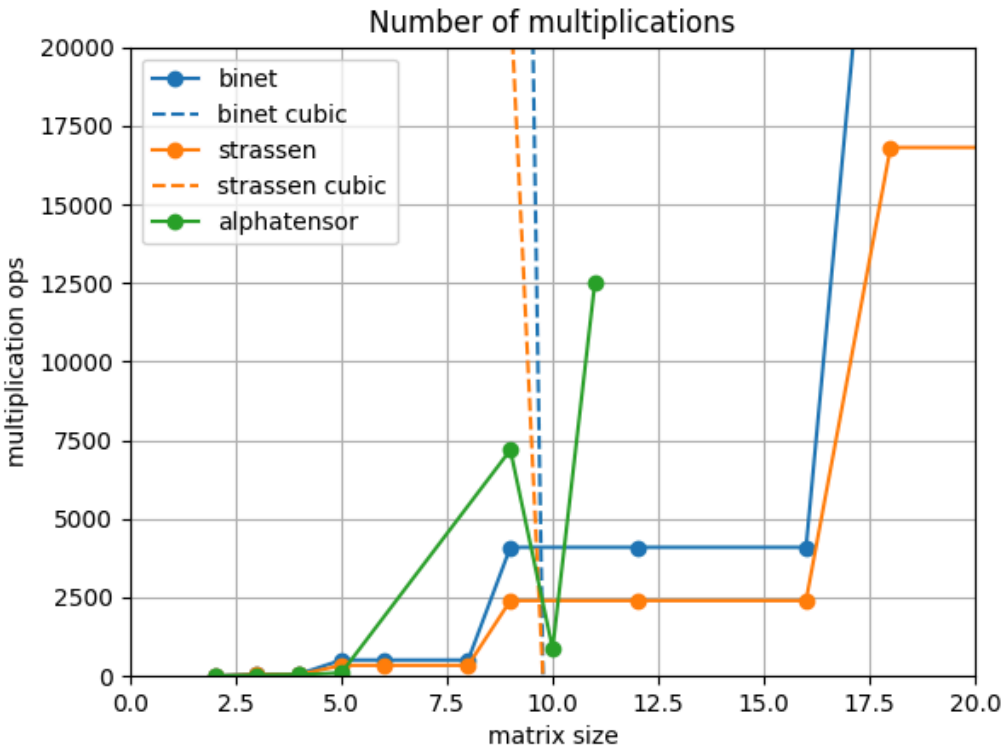
Przybliżony rysunek 1: Przybliżony wykres czasu wykonania w zależności od rozmiaru macierzy n

Przy $n = 100$ Strassen jest wyraźnie szybszy od Bineta (zob. Przybliżony rysunek 1). Skoki czasu wynikają z dopełniania do potęgi dwójki: na przykład 50 i 100 są liczone jako 64 i 128.

Liczba operacji mnożenia w zależności od rozmiaru macierzy



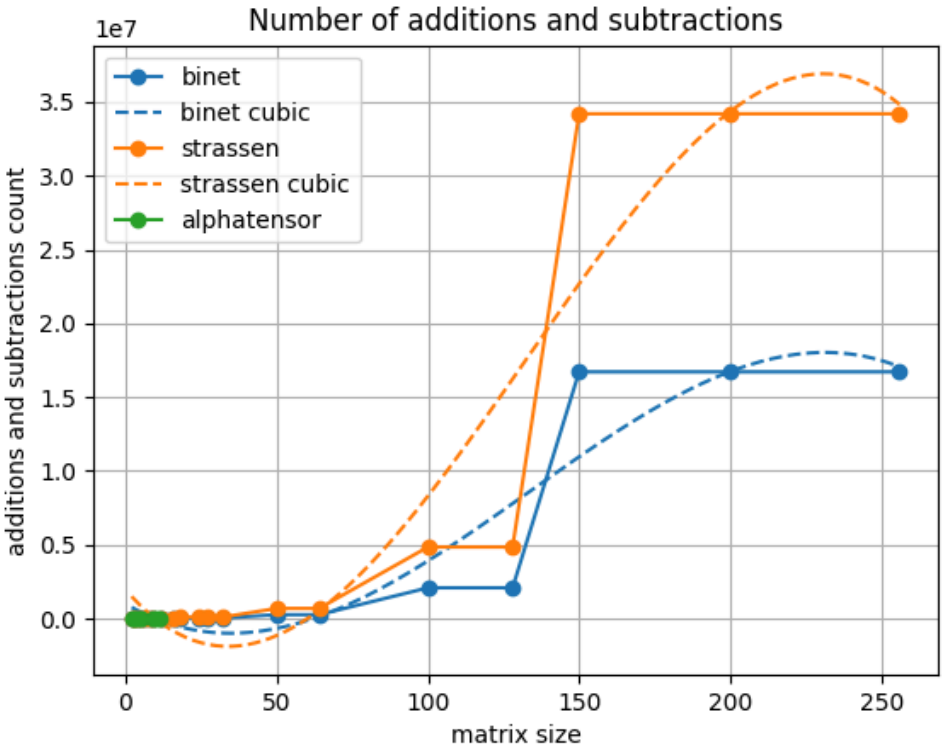
Rysunek 2: Liczba operacji mnożenia w zależności od rozmiaru macierzy n



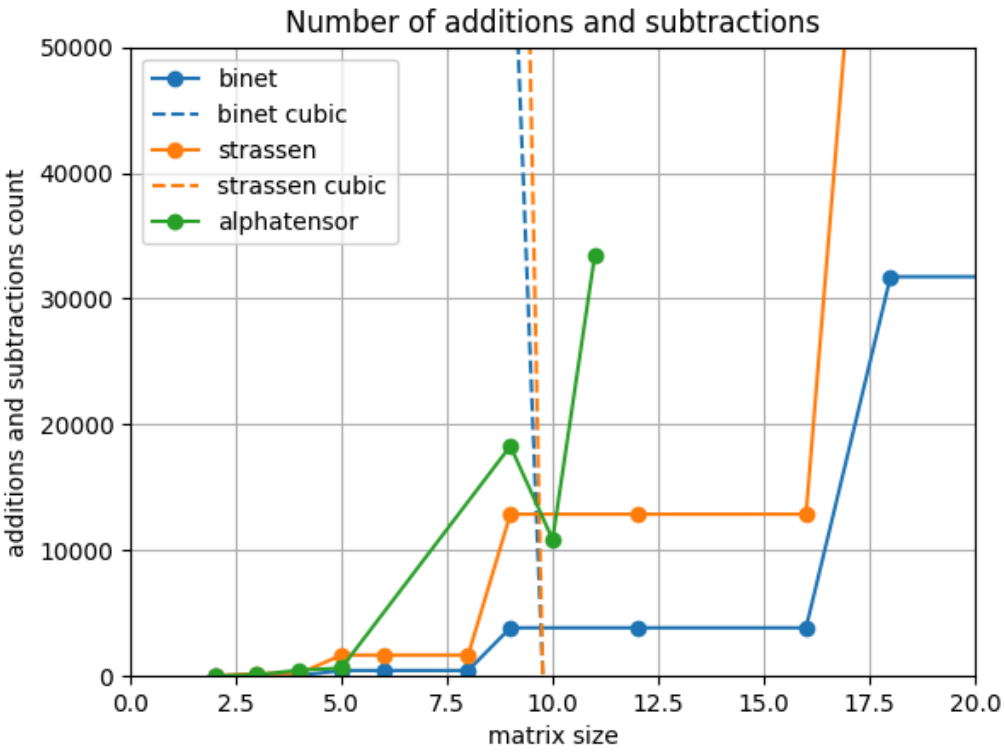
Przybliżony rysunek 2: Przybliżony wykres liczby operacji mnożenia w zależności od rozmiaru macierzy n

Dla $n = 100$ (po dopełnieniu do 128) Binet wykonuje $128^3 = 2\,097\,152$ mnożeń, a Strassen $7^7 = 823\,543$ (zob. Przybliżony rysunek 2).

Liczba operacji dodawania i odejmowania w zależności od rozmiaru macierzy



Rysunek 3: Liczba operacji dodawania i odejmowania w zależności od rozmiaru macierzy n



Przybliżony rysunek 3: Przybliżony wykres liczby operacji dodawania i odejmowania w zależności od rozmiaru macierzy n

Krzywa Strassena leży wyraźnie wyżej (zob. Przybliżony rysunek 3). To koszt siedmiu iloczynów złożonych z sum i różnic bloków. Operacje zmiennoprzecinkowe to suma tych dwóch wykresów: Strassen ma ich łącznie więcej niż Binet, bo oszczędność na mnożeniach nie równoważy dodatkowych dodawań. Mimo to jest szybszy, bo mnożenia skalarne w tej implementacji ważą więcej niż dodawania.

Porównanie dokładności obliczeń

Dokładność obu algorytmów rekurencyjnych oraz algorytmu AlphaTensor sprawdziliśmy, porównując ich wyniki z wynikiem mnożenia w GNU Octave 8.4.0. Test wykonano na losowych macierzach $A, B \in \mathbb{R}^{4\times 4}$, a iloczyn obliczony w Octave, $C^{\mathrm{Oct}} = AB$, potraktowano jako wartość odniesienia.

Dla wyniku C uzyskanego daną metodą obliczono dwie miary błędu, w obu przypadkach jako maksimum po wszystkich elementach macierzy:

$$E_{\mathrm{abs}} = \max_{i,j} |C_{ij} - C_{ij}^{\mathrm{Oct}}|,$$

$$E_{\mathrm{rel}} = \max_{i,j} |C_{ij} - C_{ij}^{\mathrm{Oct}}| \frac{1}{|C_{ij}^{\mathrm{Oct}}|} .$$

Pierwsza miara to maksymalny błąd bezwzględny, a druga maksymalny błąd względny, czyli błąd odniesiony do wartości elementu z Octave.

Metoda	Błąd bezwzględny E_{abs}	Błąd względny E_{rel}
Binet	$2,220 \cdot 10^{-16}$	$2,210 \cdot 10^{-16}$
Strassen	$6,661 \cdot 10^{-16}$	$2,279 \cdot 10^{-15}$
AlphaTensor	$1,332 \cdot 10^{-15}$	$4,177 \cdot 10^{-15}$

Tabela 1: Błędy obliczeń względem GNU Octave 8.4.0 dla losowej macierzy 4×4

Złożoność teoretyczna a eksperymentalna

Wykładnik k w zależności $M(n) \sim n^k$ dla liczby operacji mnożenia wyznaczono eksperymentalnie jako współczynnik kierunkowy prostej dopasowanej metodą najmniejszych kwadratów do zmierzonych liczb mnożeń w skali logarytmicznej. Użyto rozmiarów będących potęgami dwójki, $n \in \{4, 8, 16, 32, 64, 128, 256\}$, bo dla nich nie występuje dopełnianie, które zaburzałoby wynik. Liczba mnożeń jest dla tych rozmiarów równa dokładnie n^3 (Binet) oraz $7^{\log_2 n}$ (Strassen), dlatego błąd standardowego dopasowania wynosi $0,000$.

Metoda	Wykładnik teoretyczny	Wykładnik eksperymentalny k
Binet	3	$3,000 \pm 0,000$
Strassen	$\log_2 7 \approx 2,807$	$2,807 \pm 0,000$

Tabela 2: Teoretyczny i eksperymentalny wykładnik złożoności liczby operacji mnożenia

Kompilacja

Program można skompilować, jeśli nie chcemy uruchamiać go jako interpretowanego skryptu Pythona. W tym celu wystarczy zamienić notatnik na plik źródłowy, a następnie zbudować go narzędziem Nuitka, które tłumaczy ten plik na C i tworzy z niego samodzielny plik wykonywalny. Do linkowania na Linuksie potrzebny jest `patchelf`.

```
jupyter nbconvert --to script lab1.ipynb --output lab1.py
nuitka --standalone lab1.py
```